

FAMILY FINANCE DASHBOARD

Self-Hosted Proxmox LXC Build Plan

Actual Budget + SimpleFIN + Custom Couples / Household Layer

Target	Two-spouse household finance management
Ownership model	Me / Wife / Joint
LXC size	2 vCPU / 2 GB RAM / 1 GB swap
Disk	24-32 GB
Host	Proxmox LXC, Debian 13, Docker Compose

Build principle: Actual is the financial ledger. The custom app only adds household ownership, responsibility, reconciliation, and simplified visualization.

1. Executive Summary

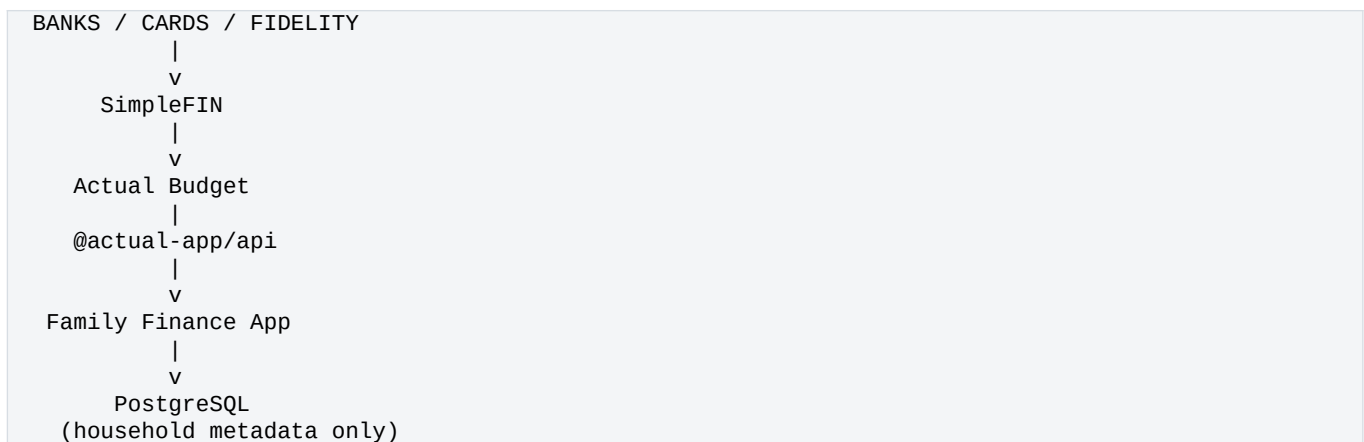
Build a self-hosted household finance system for two spouses that combines bank synchronization, budgeting, recurring bills, personal-versus-shared expenses, payment responsibility, joint expense splitting, and simple reporting. The primary UI model is always: **ME / WIFE / JOINT**

Core design rule

Track "who owns the expense" separately from "who paid it." A joint purchase can be paid from either spouse's personal card while remaining a joint household expense.

- Actual Budget is the canonical source for accounts, transactions, categories, schedules, split transactions, and budget calculations.
- SimpleFIN provides the external bank aggregation bridge for supported institutions such as Chase, Citi, and Fidelity account types where available.
- The custom Family Finance App adds household-specific metadata and views that Actual does not natively model.
- One shared household budget is used; do not create separate financial ledgers for each spouse.
- The finished system should feel like Actual Budget + Rocket Money-style bill visibility + couples expense management.

2. System Architecture



Component Responsibilities

Component	Owns	Does not own
Actual Budget	Accounts, transactions, categories, budgets, schedules, splits, financial history	Household ownership/reconciliation logic
SimpleFIN	Read-only bank aggregation bridge	Budget logic or household metadata
Family Finance App	Bills UI, Me/Wife/Joint ownership, payer, splits, reconciliation, dashboards	Bank ledger or duplicate transaction history
PostgreSQL	Household-specific metadata and app settings	A second copy of the financial ledger

3. Proxmox LXC Deployment

Recommended starting size

2 vCPU, 2 GB RAM, 1 GB swap, and 24-32 GB disk. This is appropriate for a two-user household MVP and can be increased later in Proxmox if builds or reporting need more headroom.

Resource	Recommendation	Notes
Hostname	family-finance	Dedicated LXC
OS	Debian 13	Current stable Debian is preferred
CPU	2 vCPU	Enough for two household users
RAM	2048 MB	Start here; monitor actual use
Swap	1024 MB	Safety margin for builds and updates
Disk	24-32 GB	Docker images/logs use more space than finance data
LXC	Unprivileged	Preferred security posture
Nesting	Enabled	Required for Docker-in-LXC

Suggested Docker Compose services: **actual-server**, **family-finance-web/api**, **postgres**, reverse proxy (optional if provided elsewhere).

Resource note

The likely temporary pressure point is Node/Next.js dependency installation and production builds, not normal runtime. If the LXC experiences OOM kills during builds, increase swap to 2 GB or RAM to 3-4 GB before changing the architecture.

4. Actual Budget Account Organization

Use one household Actual budget and visually group accounts by ownership. Account ownership is not the same as transaction ownership.

ME

- Personal checking
- Personal savings
- Personal credit cards
- Fidelity / CMA where appropriate

WIFE

- Personal checking
- Personal savings
- Personal credit cards
- Fidelity / CMA where appropriate

JOINT

- Joint checking
- Joint savings
- Joint credit cards

5. Household Data Model

Three independent dimensions

Transaction	Expense owner	Payer	Payment account
Gym	ME	ME	My checking
Salon	WIFE	WIFE	Wife card
Mortgage	JOINT	JOINT	Joint checking
Netflix	JOINT	ME	My card

Groceries	JOINT	WIFE	Wife Citi
-----------	-------	------	-----------

Joint split rules: 50/50, 60/40, 40/60, custom percentage, or custom dollar amount. Store a configurable default household split.

PostgreSQL entities

```

household_members
  id, name, type

transaction_metadata
  actual_transaction_id
  expense_owner
  payer
  split_rule
  notes
  review_status

bills
  id, name, actual_schedule_id, category
  expected_amount, frequency, due_date
  expense_owner, responsible_person, payer
  payment_account_id, autopay, split_rule
  active, notes

split_rules
  id, name, me_percentage, wife_percentage

household_settings
  default_joint_split, currency, timezone
  budget_month_start, notification_preferences

```

6. Transaction Classification

Use Actual categories for **what the money was for**, and custom metadata for **who it belongs to / who paid it**.

- Example categories: Housing, Utilities, Groceries, Restaurants, Transportation, Insurance, Subscriptions, Entertainment, Healthcare, Personal Spending, Travel, Home.
- Never infer expense ownership solely from the bank account that paid it.
- Allow merchant rules such as Netflix -> JOINT, my gym -> ME, wife salon -> WIFE, mortgage -> JOINT.
- New or uncertain transactions should enter a review queue rather than silently receiving a questionable classification.

Split transaction example

```

Target - $200 total
$80  Groceries    JOINT
$70  Household   JOINT
$50  Clothing    WIFE

```

Use Actual split transactions wherever possible so the original card transaction remains intact while ownership and category reporting remain accurate.

7. Bills Manager

Bills are a primary feature. Extend Actual Schedules rather than recreating a separate recurrence engine.

Bill field	Purpose
Name / amount	Expected and actual amount
Due date / frequency	When the bill is expected

Bill field	Purpose
Category	What it is for
Expense owner	ME / WIFE / JOINT
Responsible person	Who must ensure it is handled
Payer	Who normally funds it
Payment account	Actual account reference
Autopay	Yes / no
Split	Joint contribution rule
Last / next payment	Visibility and status
Status	Upcoming / Due Soon / Paid / Overdue / Autopay / Needs Review

Required bill views: **All Bills** **My Bills** **Wife's Bills** **Joint Bills** **Autopay** **Due This Week** **Due This Month** **Overdue** **Subscriptions**

8. Main Dashboard and Budget UX

The custom UI should feel like a household command center, not accounting software. The global filter is always: **ALL** | **ME** | **WIFE** | **JOINT**

SEPTEMBER		
ME	WIFE	JOINT
Spent: \$1,825	Spent: \$1,340	Spent: \$4,680
Remaining: \$975	Remaining: \$1,210	Remaining: \$1,520
UPCOMING BILLS		
[Paid] Mortgage	\$2,150	Joint
[Paid] Internet	\$85	Joint - Me paid
[Due] Electric	\$162	Joint - Sep 14
[Due] Wife Car	\$540	Wife - Sep 18
BUDGET		
Groceries	\$812 / \$900	
Restaurants	\$310 / \$500	
Entertainment	\$220 / \$300	
Gas	\$205 / \$400	

- Each Me/Wife/Joint card should show Income, Spent, Budgeted, Remaining, and Bills Remaining.
- Budget views should use readable progress bars, remaining amounts, and clear overspending indicators.
- Separate visual sections for Household, Me, and Wife.
- Mobile-first layouts and minimal clicks are more important than exposing every accounting field.

9. Joint Expense Reconciliation

Create a dedicated view answering one question: **Are our shared expenses currently balanced?**

JOINT SPENDING - SEPTEMBER

Paid by Me	\$1,425
Paid by Wife	\$1,010
Paid from Joint	\$3,220
Target split	50 / 50
Expected contribution	\$2,827.50 each
Actual effective contribution	
Me	\$2,925
Wife	\$2,730
Difference	
Joint owes Me	\$97.50
Wife under target	\$97.50

Support alternate household contribution models such as 60/40 and custom percentages. Calculations should account for payments made directly from joint accounts as well as personal accounts used for joint expenses.

10. Transaction Review and Reports

Review queue

- Show date, merchant, amount, account, category, expense owner, payer, and split.
- Provide one-tap ownership assignment: ME / WIFE / JOINT.
- Support bulk classification and reusable merchant rules.
- Flag imported items that have no ownership metadata as Needs Review.

Household reports

- Monthly household spending; My spending; Wife spending; Joint spending.
- Bills by person; Joint contributions; Spending by category and account.
- Income vs spending; Net cash flow; Budget vs actual; Subscription totals.
- 12-month trends and month-over-month comparisons.

Do not duplicate every native Actual report. Prioritize reports that depend on the household ownership model.

11. Application UX, Authentication, and Access

Primary navigation: **Dashboard Bills Budget Transactions Joint Reports Settings**

- Both spouses have separate logins.
- Initial roles: Admin and Household Member.
- Both spouses may view household finances unless finer permissions are added later.
- Prefer an existing homelab OIDC provider such as Authentik, Authelia, or Keycloak when available.
- Ideally Actual and the custom app use the same identity provider.

12. Security and Backup Requirements

- HTTPS only. Do not expose the finance applications directly to the public Internet.
- Prefer VPN, Tailscale, or WireGuard for remote access.
- Use an unprivileged LXC and restrict container access.
- Keep database and SimpleFIN credentials server-side in protected environment/secrets files.

- Do not commit secrets to Git. PostgreSQL should not be externally reachable.
- Back up Actual data, PostgreSQL, Compose files, and configuration.
- Encrypt backup destinations and periodically test a full restore.

Backup item	Suggested policy
Application / data backup	Daily
Proxmox / PBS backup	Daily
Retained snapshot	Weekly
Long-term retained backup	Monthly
Restore test	Periodic / documented

13. Recommended Technology Stack

Layer	Recommendation
Frontend	Next.js + TypeScript + Tailwind
Backend	Node.js + TypeScript
Actual integration	@actual-app/api
App database	PostgreSQL
Deployment	Docker Compose
Host	Proxmox LXC
Bank aggregation	SimpleFIN
Reverse proxy	Caddy / Traefik / existing homelab proxy
Authentication	Existing OIDC provider if available

Architecture preference

Keep this as one straightforward application stack. Avoid microservices unless a real operational need appears.

14. Implementation Phases

Phase 1 - Infrastructure: Create LXC; install Docker; persistent storage; Actual; backups; HTTPS/internal DNS; SimpleFIN; import accounts.

Phase 2 - Actual Configuration: Create household budget; categories; schedules; normalize account names; test transaction synchronization.

Phase 3 - Custom App Foundation: Create app; connect @actual-app/api; PostgreSQL schema; authentication; read Actual accounts/transactions/categories/schedules.

Phase 4 - Ownership Model: Implement ME/WIFE/JOINT; expense owner; payer; split rules; transaction metadata; classification defaults.

Phase 5 - Bills: Bill list; upcoming bills; due dates; autopay; responsible person; payment account; paid/unpaid state.

Phase 6 - Dashboard: Monthly overview; Me/Wife/Joint cards; budget progress; upcoming bills; recent transactions; review warning.

Phase 7 - Joint Reconciliation: Contribution totals; split calculations; balance owed; monthly history.

Phase 8 - Polish: Mobile UI; reports; merchant rules; bulk classification; charts; search/filtering; optional notifications.

15. MVP Acceptance Checklist

- ☐ Actual synchronizes financial transactions successfully.
- ☐ Both spouses can log in.

- ☐ Accounts appear correctly and remain organized.
- ☐ Every transaction can be labeled ME, WIFE, or JOINT.
- ☐ Payer is tracked independently from expense owner.
- ☐ Joint expenses support a configurable split.
- ☐ Bills show due dates, responsibility, payment account, and status.
- ☐ Dashboard shows Me/Wife/Joint spending and budget progress.
- ☐ Joint contribution calculations work correctly.
- ☐ Data survives container and application restarts.
- ☐ Backups are running and a restore procedure is documented.

16. Agent Handoff Rules

Do not rebuild Actual Budget

Actual remains responsible for accounts, transactions, bank imports, financial ledger, categories, budget calculations, schedules, and split transactions. The custom app exists only to add the household layer.

CUSTOM APP MUST ANSWER:

Who owns it?
 Who paid it?
 Is it joint?
 How should it be split?
 Who is responsible for the bill?
 Are we balanced?
 What is due next?

FINAL TARGET

**Actual Budget + Rocket Money-style bill visibility + Couples expense management +
 Household dashboard + Me / Wife / Joint accounting**

Self-hosted in the Proxmox homelab except for the external bank aggregation service required to retrieve transactions.